

Multiliquid
Security Review

Cantina Managed review by:

Desmond Ho, Lead Security Researcher
Rvierdiiev, Security Researcher

September 2, 2025

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
- 3.1 Medium Risk . . . 4
  - 3.1.1 Issuer admin can transfer user funds from StablecoinDelegate approvals . . . 4
  - 3.1.2 RWA redemption fees are swapped to RWA and sent to user . . . 4
- 3.2 Low Risk . . . 4
  - 3.2.1 Repeated scaling in RWA and stablecoin amount conversion view functions . . . 4
- 3.3 Gas Optimization . . . 5
  - 3.3.1 `stablecoinAddress` can be cached in memory . . . 5
  - 3.3.2 Duplicate rate check . . . 7
- 3.4 Informational . . . 7
  - 3.4.1 Comment Clarification . . . 7
  - 3.4.2 Absent checks in calculation methods . . . 7
  - 3.4.3 Add slippage protection to swaps . . . 7

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Multiliquid enables seamless, 24/7 exchanges of blue-chip real-world assets (RWAs) and stablecoins, across multiple blockchains, at scale.

From Aug 20th to Aug 26th the Cantina team conducted a review of Multiliquid on commit hash a1f968bc. The team identified a total of **8** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	1	1	0
Gas Optimizations	2	2	0
Informational	3	3	0
Total	**8**	**8**	**0**

3 Findings

3.1 Medium Risk

3.1.1 Issuer admin can transfer user funds from StablecoinDelegate approvals

Severity: Medium Risk

Context: StablecoinDelegate.sol#L202-L205

Description: To interact with `StablecoinDelegate`, a user must provide an allowance for the stablecoin, which can be set to an infinite amount. The `depositStablecoin()` function allows the issuer admin to transfer stablecoins from any user who has approved the delegate contract. Additionally, the `withdrawStablecoin()` function enables withdrawing those funds.

This design introduces unnecessary risk, as it grants the issuer admin broad control over user funds once approval is given.

Recommendation: Project comments indicate that this functionality is not required. It is recommended to remove `depositStablecoin()` function to reduce risk.

Multiliquid: Fixed in commit b19a1f26.

Cantina Managed: Fix verified.

3.1.2 RWA redemption fees are swapped to RWA and sent to user

Severity: Medium Risk

Context: AUSDDelegate.sol#L233

Description: When swapping from a stablecoin to RWA, `stablecoinToSwap` will include the redemption fee, which shouldn't be the case, since fees are expected to be collected in stablecoin. It is then swapped to the RWA and sent to the user, which means the redemption fee isn't actually taken.

Proof of Concept:

```
function test_redemptionFeeNotRetained() public {
    // Fast forward time by 7 days to avoid arithmetic underflow in ULTRADelegate
    vm.warp(block.timestamp + 7 days);
    setupAgoraPools();

    // set redemption fee to 1%
    vm.prank(stablecoinIssuerAdmin3);
    ausdDelegate.setRWARedemptionFee(ULTRA_ID, 1e16);

    //Mint AUSD to user
    ausd.mint(regularUser, 150_000e6); //150k AUSD

    vm.startPrank(regularUser);
    //Approval to the ausd delegate
    ausd.approve(address(ausdDelegate), type(uint256).max);

    //Swap from AUSD to Ultra
    multiliquidSwap.swapIntoRWA(ULTRA_ID, AUSD_ID, 50_000e6, "", "");

    // assert that delegate has not kept redemption fee in either RWA or AUSD
    assertEq(ausd.balanceOf(address(ausdDelegate)), 0);
    assertEq(ultra.balanceOf(address(ausdDelegate)), 0);
}
```

Recommendation: Pass the redemption fee to this function and subtract it from `stablecoinToSwap`.

Multiliquid: Fixed in commit 2f9ed0c5.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Repeated scaling in RWA and stablecoin amount conversion view functions

Severity: Low Risk

Context: MultiliquidSwap.sol#L397-L399, MultiliquidSwap.sol#L409-L412, MultiliquidSwap.sol#L426-L428, MultiliquidSwap.sol#L432-L435

Description: The external view functions `calculateRWAAmt()` and `calculateStableAmt()` are performing redundant decimal scaling operations that are already handled internally by their respective helper functions `_calculateRWAAmtAndFeeAmt()` and `_calculateStableAndFeeAmt()`.

Proof of Concept:

```
function test_calculateStableAmt_RepeatedRounding() public {
    address mockStablecoin = makeAddr("mockStablecoin");
    swap.setStablecoinAcceptance(
        stablecoinID, address(stablecoinDelegate), mockStablecoin, 6, true, false
    );
    uint256 rwaValue = 3e18; // 3 USD
    uint256 rwaAmount = 100e18; // 100 RWA
    uint256 adjustmentBps = 20e14; // 20 bps
    uint256 feeBps = 10e14; // 10 bps
    uint256 redemptionFeeBps = 0; // No redemption fee for this test

    _setupScenario(rwaValue, adjustmentBps, feeBps, redemptionFeeBps);

    (uint256 customerAmount, uint256 feeAmount) =
        swap.calculateStableAmt(stablecoinID, rwaID, rwaAmount, "");
    assertEq(customerAmount, 0);
    assertEq(feeAmount, 0);
}
```

Recommendation: Remove the scaling operations in the view functions.

Multiliquid: Fixed in commit 34acdfd2.

Cantina Managed: Fix verified.

3.3 Gas Optimization

3.3.1 `stablecoinAddress` can be cached in memory

Severity: Gas Optimization

Context: AUSDDelegate.sol#L205-L222, AUSDDelegate.sol#L230-L240

Description: `stablecoinAddress` can be cached and casted in memory to avoid repeated storage reads:

```
test_maliciousRWA_RevertTransfer() (gas: 10 (0.001%))
test_deployStablecoin_AgoraSwapReverts() (gas: 10 (0.006%))
test_deployStablecoin_InsufficientAUSDOutput() (gas: 10 (0.008%))
test_maliciousStablecoinToRwa_ReducedRwaOutput() (gas: -395 (-0.013%))
test_maliciousAgoraContract_ReducedOutput() (gas: -395 (-0.013%))
test_maliciousAgoraContract_ZeroOutput() (gas: -395 (-0.014%))
test_MegaE2E() (gas: -1056 (-0.058%))
test_fuzz_normal_swap_into_rwa_happy_path_ausd(uint256) (gas: -393 (-0.071%))
test_normal_swap_into_rwa_happy_path_ausd() (gas: -393 (-0.072%))
test_normal_swap_into_stablecoin_happy_path_ausd() (gas: -663 (-0.124%))
test_receiveStablecoin_minimal() (gas: -393 (-0.124%))
test_normal_swap_into_stablecoin_happy_path_fuzz(uint256) (gas: -663 (-0.130%))
test_receiveStablecoin_Success() (gas: -393 (-0.145%))
test_receiveStablecoin_InsufficientRWAAmount() (gas: -260 (-0.154%))
test_deployStablecoin_Success_RWAAsToken0() (gas: -663 (-0.237%))
test_deployStablecoin_ReentrancyProtection() (gas: -663 (-0.256%))
test_receiveStablecoin_ZeroAmounts() (gas: -393 (-0.307%))
test_maliciousRWA_TransferFee_RwaToStablecoin() (gas: -50943 (-0.601%))
test_deployStablecoin_InvalidTokenConfiguration() (gas: -51213 (-0.602%))
test_deployStablecoin_Success_RWAAsToken1() (gas: -51615 (-0.667%))
test_deployStablecoin_InvalidAgoraContract() (gas: -50942 (-0.685%))
test_receiveStablecoin_InvalidAgoraContract() (gas: -51212 (-0.687%))
test_maliciousStablecoin_TransferFee_StablecoinToRwa() (gas: -51212 (-0.887%))
test_maliciousStablecoin_BalanceManipulation() (gas: -50942 (-0.890%))
Overall gas change: -365167 (-0.210%)
```

Recommendation: Apply the following patch:

```
diff --git a/src/ProtocolStablecoinDelegates/AUSDDelegate.sol
↪  b/src/ProtocolStablecoinDelegates/AUSDDelegate.sol
index 838a5d6..bf1f5c5 100644
-- a/src/ProtocolStablecoinDelegates/AUSDDelegate.sol
++ b/src/ProtocolStablecoinDelegates/AUSDDelegate.sol
@@ -187,9 +187,10 @@ contract AUSDDelegate is StablecoinDelegate, IExternalSwapStablecoinDelegate {
            AgoraStableSwapPair(agoraStableSwapContract[params.rwaAddress]);
        if (address(agoraContract) == address(0)) revert InvalidAgoraContract();

+       IERC20 _stablecoin = IERC20(stablecoinAddress);
        // Calculate expected AUSD amount
        uint256 expectedAusdAmount = _getExpectedOutput(
-           address(agoraContract), stablecoinAddress, params.rwaAddress, params.rwaAmount
+           address(agoraContract), address(_stablecoin), params.rwaAddress, params.rwaAmount
        );

        // Ensure we get enough AUSD to cover vault fee + user amount
@@ -203,13 +204,13 @@ contract AUSDDelegate is StablecoinDelegate, IExternalSwapStablecoinDelegate {
        );

        // Check balance before swap
-       uint256 balanceBefore = IERC20(stablecoinAddress).balanceOf(address(this));
+       uint256 balanceBefore = _stablecoin.balanceOf(address(this));

        // Execute the swap
-       _performSwap(agoraContract, expectedAusdAmount, address(this), stablecoinAddress);
+       _performSwap(agoraContract, expectedAusdAmount, address(this), address(_stablecoin));

        // Verify actual amount received
-       uint256 balanceAfter = IERC20(stablecoinAddress).balanceOf(address(this));
+       uint256 balanceAfter = _stablecoin.balanceOf(address(this));
        uint256 actualReceived = balanceAfter - balanceBefore;
        uint256 totalRequired = params.vaultAmount + params.userAmount;

@@ -218,8 +219,8 @@ contract AUSDDelegate is StablecoinDelegate, IExternalSwapStablecoinDelegate {
        }

        // Distribute the received AUSD
-       IERC20(stablecoinAddress).safeTransfer(params.vault, params.vaultAmount);
-       IERC20(stablecoinAddress).safeTransfer(params.user, params.userAmount);
+       _stablecoin.safeTransfer(params.vault, params.vaultAmount);
+       _stablecoin.safeTransfer(params.user, params.userAmount);
    }

    function _executeStablecoinToRwaSwap(SwapIntoRwaParams memory params) internal {
@@ -227,17 +228,18 @@ contract AUSDDelegate is StablecoinDelegate, IExternalSwapStablecoinDelegate {
            AgoraStableSwapPair(agoraStableSwapContract[params.rwaAddress]);
        if (address(agoraContract) == address(0)) revert InvalidAgoraContract();

-       IERC20(stablecoinAddress).safeTransferFrom(params.user, address(this), params.totalAmount);
-       IERC20(stablecoinAddress).safeTransfer(params.vault, params.vaultAmount);
+       IERC20 _stablecoin = IERC20(stablecoinAddress);
+       _stablecoin.safeTransferFrom(params.user, address(this), params.totalAmount);
+       _stablecoin.safeTransfer(params.vault, params.vaultAmount);

        uint256 stablecoinToSwap = params.totalAmount - params.vaultAmount;
        uint256 expectedRwaAmount = _getExpectedOutput(
-           address(agoraContract), params.rwaAddress, stablecoinAddress, stablecoinToSwap
+           address(agoraContract), params.rwaAddress, address(_stablecoin), stablecoinToSwap
        );

        if (expectedRwaAmount < params.rwaAmount) revert InsufficientRWAAmount();

-       IERC20(stablecoinAddress).safeTransfer(address(agoraContract), stablecoinToSwap);
+       _stablecoin.safeTransfer(address(agoraContract), stablecoinToSwap);

        // Check user's RWA balance before swap
        uint256 userRwaBalanceBefore = IERC20(params.rwaAddress).balanceOf(params.user);
```

Multiliquid: Fixed in commit 34393c8d.

Cantina Managed: Fix verified.

3.3.2 Duplicate rate check

Severity: Gas Optimization

Context: StablecoinDelegate.sol#L188, StablecoinDelegate.sol#L195

Description: The referenced checks are repeated; it'll be done in `multliquidSwap`.

Recommendation: Remove the referenced lines.

Multiliquid: Fixed in commit 46523b3c.

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Comment Clarification

Severity: Informational

Context: ULTRADelegate.sol#L101

Description: `0 to disable` may give the impression that it disables the volume limit, but it means disable swaps entirely. Recommend clarifying the comment.

Recommendation:

```
- 0 to disable
+ 0 to disable swaps
```

Multiliquid: Fixed in commit 34393c8d.

Cantina Managed: Fix verified.

3.4.2 Absent checks in calculation methods

Severity: Informational

Context: MultiliquidSwap.sol#L388

Description: The checks that exist in he swap functions are absent in these calculation methods.

Recommendation: Consider adding the checks into these functions. In addition, since there is significant overlap between the calculation and swap functions, they can be refactored and abstracted into internal functions to avoid code duplication.

Multiliquid: Fixed in commit 7a691864.

Cantina Managed: Fix verified.

3.4.3 Add slippage protection to swaps

Severity: Informational

Context: MultiliquidSwap.sol#L232-L238, MultiliquidSwap.sol#L314-L320

Description/Recommendation: Consider adding a `minStablecoinAmt` to prevent the swaps to be executed at undesirable rates, and from protection against unexpected changes to redemption fees and discounts by issuer admins right before the swap transaction.

Multiliquid: Fixed in commit 8492e543.

Cantina Managed: Fix verified.